

TABLE DES MATIÈRES

PARTIE I	– FONDEMENTS : ML VS DL, PHILOSOPHIE ET OUTILLAGE
PARTIE II	– SCIKIT-LEARN : MACHINE LEARNING CLASSIQUE
PARTIE III	– KERAS & TENSORFLOW : FONDEMENTS DU DEEP LEARNING
PARTIE IV	– ALGORITHMES NATIFS DEEP LEARNING
PARTIE V	– RÉSEAUX DE NEURONES CONVOLUTIFS (CNN)
PARTIE VI	– RÉSEAUX RÉCURRENTS (RNN, LSTM, GRU)
PARTIE VII	– GENERATIVE ADVERSARIAL NETWORKS (GAN)
PARTIE VIII	– LATENT DIFFUSION MODELS (LDM)
PARTIE IX	– TRANSFORMERS ET LARGE LANGUAGE MODELS (LLM)
PARTIE X	– PIPELINES, DÉPLOIEMENT ET BONNES PRATIQUES
PARTIE XI	– PROJETS COMPLETS COMMENTÉS

=====

PARTIE I – FONDEMENTS : ML VS DL, PHILOSOPHIE ET OUTILLAGE

=====

1.1 MACHINE LEARNING VS DEEP LEARNING : QUELLE DIFFÉRENCE ?

Le Machine Learning (ML) et le Deep Learning (DL) sont deux branches de l'Intelligence Artificielle. Comprendre leur différence fondamentale est indispensable pour choisir le bon outil face à un problème.

MACHINE LEARNING

- L'algorithme apprend à partir de features (caractéristiques) choisies manuellement par l'expert.
- Le feature engineering est central : c'est l'humain qui décide quelles variables donner au modèle.
- Fonctionne très bien sur des données tabulaires structurées.
- Interprétable, rapide à entraîner, nécessite moins de données.
- Outils : scikit-learn, XGBoost, LightGBM, CatBoost.

DEEP LEARNING

- L'algorithme apprend lui-même les features à partir des données brutes via des couches de neurones empilées.
- Pas (ou peu) de feature engineering : on donne l'image, le texte, le son brut.
- Excelle sur données non structurées : images, texte, audio, vidéo.
- Nécessite de grandes quantités de données et de puissance de calcul (GPU).
- Outils : Keras, PyTorch, TensorFlow.

RÈGLE D'OR :

- Données tabulaires, peu de données, besoin d'interprétabilité → ML classique
- Images, texte, séquences, gros volumes → Deep Learning

1.2 LES TROIS TYPES D'APPRENTISSAGE

[A] APPRENTISSAGE SUPERVISÉ

- On fournit des données étiquetées ($X \rightarrow y$ connue).
- Le modèle apprend à prédire y à partir de X .
- Exemples : classification, régression, détection d'objets.

[B] APPRENTISSAGE NON SUPERVISÉ

- Pas d'étiquettes. Le modèle découvre des structures cachées.
- Exemples : clustering, réduction de dimension, génération.

[C] APPRENTISSAGE PAR RENFORCEMENT

- Un agent interagit avec un environnement et apprend par récompenses et pénalités.

- Exemples : jeux vidéo, robotique, trading algorithmique.

1.3 ENVIRONNEMENT ET OUTILLAGE PYTHON

INSTALLATION RECOMMANDÉE :

```
# Créer un environnement virtuel dédié
python -m venv env_ml
source env_ml/bin/activate # Linux/Mac
env_ml\Scripts\activate    # Windows

# Installer les bibliothèques essentielles
pip install numpy pandas matplotlib seaborn
pip install scikit-learn
pip install tensorflow keras
pip install torch torchvision # optionnel si PyTorch préféré
```

IMPORTS DE BASE :

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
```

VÉRIFICATION GPU :

```
import tensorflow as tf
print(tf.config.list_physical_devices('GPU'))
# Si liste non vide → GPU disponible et utilisable
```

1.4 LE PIPELINE UNIVERSEL D'UN PROJET ML/DL

```
ÉTAPE 1 – Définir le problème (classification ? régression ? génération ?)
ÉTAPE 2 – Collecter et explorer les données (EDA)
ÉTAPE 3 – Préparer les données (nettoyage, normalisation, split)
ÉTAPE 4 – Choisir et construire le modèle
ÉTAPE 5 – Entraîner le modèle
ÉTAPE 6 – Évaluer et valider (métriques, courbes)
ÉTAPE 7 – Optimiser (hyperparamètres, architecture)
ÉTAPE 8 – Déployer et monitorer
```

Ce pipeline s'applique aussi bien à un Random Forest qu'à un Transformer.
La rigueur de chaque étape détermine la qualité du résultat final.

PARTIE II – SCIKIT-LEARN : MACHINE LEARNING CLASSIQUE

2.1 PHILOSOPHIE DE SCIKIT-LEARN

Scikit-learn (sklearn) est la bibliothèque de référence pour le Machine Learning en Python. Sa philosophie repose sur trois principes :

- COHÉRENCE : tous les modèles partagent la même API (fit / predict / score)
- MODULARITÉ : chaque composant est interchangeable dans un pipeline
- DOCUMENTATION : l'une des meilleures de l'écosystème Python

L'API universelle sklearn :

```
from sklearn.xxx import MonModele
```

```

modele = MonModele(hyperparametre=valeur)
modele.fit(X_train, y_train)      # Entraînement
predictions = modele.predict(X_test) # Prédiction
score = modele.score(X_test, y_test) # Évaluation

```

2.2 PRÉPARATION DES DONNÉES AVEC SKLEARN

SPLIT TRAIN/TEST :

```

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,      # 20% pour le test
    random_state=42,    # reproductibilité
    stratify=y,         # maintenir les proportions de classes
)

```

NORMALISATION :

```

from sklearn.preprocessing import StandardScaler, MinMaxScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train) # fit ET transform
X_test_scaled = scaler.transform(X_test)      # transform SEULEMENT

# PIÈGE CLASSIQUE : ne jamais faire fit sur X_test → fuite de données

```

ENCODAGE DES VARIABLES CATÉGORIELLES :

```

from sklearn.preprocessing import LabelEncoder, OneHotEncoder

# LabelEncoder : pour les labels (y)
le = LabelEncoder()
y_encoded = le.fit_transform(y)

# OneHotEncoder : pour les features (X)
from sklearn.preprocessing import OneHotEncoder
ohe = OneHotEncoder(sparse_output=False)
X_cat_encoded = ohe.fit_transform(X_cat)

```

GESTION DES VALEURS MANQUANTES :

```

from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy='mean') # ou 'median', 'most_frequent'
X_imputed = imputer.fit_transform(X)

```

2.3 ALGORITHMES DE CLASSIFICATION

RÉGRESSION LOGISTIQUE (baseline incontournable) :

```

from sklearn.linear_model import LogisticRegression

modele = LogisticRegression(max_iter=1000, C=1.0)
modele.fit(X_train, y_train)
y_pred = modele.predict(X_test)

# Interprétation des coefficients
print(modele.coef_)      # poids de chaque feature
print(modele.intercept_) # biais

```

QUAND UTILISER : baseline rapide, données linéairement séparables,
interprétabilité requise.

SUPPORT VECTOR MACHINE (SVM) :

```

from sklearn.svm import SVC

```

```

modele = SVC(kernel='rbf', C=1.0, gamma='scale', probability=True)
modele.fit(X_train, y_train)
y_pred = modele.predict(X_test)
y_proba = modele.predict_proba(X_test) # probabilités

```

KERNELS DISPONIBLES :

- 'linear' : séparation linéaire, rapide
- 'rbf' : (Radial Basis Function) non-linéaire, polyvalent
- 'poly' : polynômial, utile pour relations complexes
- 'sigmoid' : similaire à un réseau de neurones simple

QUAND UTILISER : datasets petits à moyens, haute dimension, marges claires.

K-NEAREST NEIGHBORS (KNN) :

```

from sklearn.neighbors import KNeighborsClassifier

modele = KNeighborsClassifier(n_neighbors=5, metric='euclidean')
modele.fit(X_train, y_train)
y_pred = modele.predict(X_test)

CONSEIL : tester plusieurs valeurs de k (3, 5, 7, 11) et valider.
ATTENTION : pas de phase d'entraînement réelle (lazy learner) →
            lent à la prédiction sur gros datasets.

```

ARBRES DE DÉCISION :

```

from sklearn.tree import DecisionTreeClassifier
from sklearn import tree
import matplotlib.pyplot as plt

modele = DecisionTreeClassifier(
    max_depth=5,
    min_samples_split=10,
    criterion='gini' # ou 'entropy'
)
modele.fit(X_train, y_train)

# Visualisation de l'arbre
plt.figure(figsize=(20, 10))
tree.plot_tree(modele, feature_names=features, class_names=classes,
               filled=True)
plt.show()

QUAND UTILISER : interprétabilité maximale, règles métier nécessaires.
ATTENTION : sur-apprentissage facile → limiter max_depth.

```

RANDOM FOREST (ensemble d'arbres) :

```

from sklearn.ensemble import RandomForestClassifier

modele = RandomForestClassifier(
    n_estimators=200,      # nombre d'arbres
    max_depth=10,
    max_features='sqrt',  # features par arbre
    min_samples_leaf=5,
    n_jobs=-1,            # utiliser tous les coeurs CPU
    random_state=42
)
modele.fit(X_train, y_train)

# Importance des features
importances = modele.feature_importances_
indices = np.argsort(importances)[::-1]
plt.bar(range(X.shape[1]), importances[indices])
plt.xticks(range(X.shape[1]), [features[i] for i in indices], rotation=45)
plt.title("Importance des features – Random Forest")
plt.show()

```

QUAND UTILISER : données tabulaires, robustesse, feature importance.

GRADIENT BOOSTING (XGBoost / sklearn) :

```
from sklearn.ensemble import GradientBoostingClassifier

modele = GradientBoostingClassifier(
    n_estimators=300,
    learning_rate=0.05,
    max_depth=4,
    subsample=0.8
)
modele.fit(X_train, y_train)

# Version XGBoost (plus rapide, plus puissante)
# pip install xgboost
from xgboost import XGBClassifier

modele_xgb = XGBClassifier(
    n_estimators=300,
    learning_rate=0.05,
    max_depth=4,
    subsample=0.8,
    use_label_encoder=False,
    eval_metric='logloss'
)
modele_xgb.fit(X_train, y_train,
               eval_set=[(X_test, y_test)],
               early_stopping_rounds=20,
               verbose=50)
```

QUAND UTILISER : compétitions ML, performances maximales sur tabulaire.

2.4 ALGORITHMES DE RÉGRESSION

RÉGRESSION LINÉAIRE :

```
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet

# Régression linéaire simple
modele = LinearRegression()
modele.fit(X_train, y_train)
y_pred = modele.predict(X_test)

# Ridge (régularisation L2) – pour multicollinéarité
ridge = Ridge(alpha=1.0)

# Lasso (régularisation L1) – pour sélection de features
lasso = Lasso(alpha=0.1)

# ElasticNet (L1 + L2)
elastic = ElasticNet(alpha=0.1, l1_ratio=0.5)
```

MÉTRIQUES DE RÉGRESSION :

```
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"RMSE : {rmse:.4f}")
print(f"MAE : {mae:.4f}")
print(f"R² : {r2:.4f}") # 1.0 = parfait, 0 = nul, <0 = pire que moyenne
```

2.5 ALGORITHMES DE CLUSTERING (NON SUPERVISÉ)

K-MEANS :

```
from sklearn.cluster import KMeans

# Méthode du coude pour choisir k
inerties = []
for k in range(1, 11):
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X)
    inerties.append(km.inertia_)

plt.plot(range(1, 11), inerties, 'bo-')
plt.xlabel('Nombre de clusters k')
plt.ylabel('Inertie')
plt.title('Méthode du coude')
plt.show()

# Entraînement final
km = KMeans(n_clusters=3, random_state=42, n_init=10)
labels = km.fit_predict(X)
```

DBSCAN (clusters de formes arbitraires) :

```
from sklearn.cluster import DBSCAN

dbscan = DBSCAN(eps=0.5, min_samples=5)
labels = dbscan.fit_predict(X)
# labels == -1 → points aberrants (noise)
```

RÉDUCTION DE DIMENSION :

```
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE

# PCA : linéaire, rapide
pca = PCA(n_components=2)
X_2d = pca.fit_transform(X)
print(f"Variance expliquée : {pca.explained_variance_ratio_.sum():.2%}")

# t-SNE : non-linéaire, pour visualisation uniquement
tsne = TSNE(n_components=2, random_state=42, perplexity=30)
X_tsne = tsne.fit_transform(X)
```

2.6 ÉVALUATION ET VALIDATION DES MODÈLES

MÉTRIQUES DE CLASSIFICATION :

```
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, roc_auc_score, confusion_matrix,
                             classification_report)

print(classification_report(y_test, y_pred))

# Matrice de confusion
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel('Prédit')
plt.ylabel('Réel')
plt.title('Matrice de confusion')
plt.show()

# Courbe ROC
from sklearn.metrics import roc_curve
fpr, tpr, thresholds = roc_curve(y_test, y_proba[:, 1])
plt.plot(fpr, tpr, label=f'AUC = {roc_auc_score(y_test, y_proba[:, 1]):.3f}')
plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel('Taux faux positifs')
plt.ylabel('Taux vrais positifs')
plt.title('Courbe ROC')
```

```
plt.legend()
plt.show()
```

VALIDATION CROISÉE :

```
from sklearn.model_selection import cross_val_score, StratifiedKFold

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(modele, X, y, cv=cv, scoring='f1_macro')

print(f"F1 moyen : {scores.mean():.4f}")
print(f"Écart-type : {scores.std():.4f}")
```

OPTIMISATION DES HYPERPARAMÈTRES :

```
from sklearn.model_selection import GridSearchCV, RandomizedSearchCV

# GridSearchCV : exhaustif
param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [5, 10, None],
    'max_features': ['sqrt', 'log2']
}
grid = GridSearchCV(RandomForestClassifier(), param_grid, cv=5,
                    scoring='f1_macro', n_jobs=-1)
grid.fit(X_train, y_train)
print(f"Meilleurs params : {grid.best_params_}")
print(f"Meilleur score : {grid.best_score_: .4f}")

# RandomizedSearchCV : plus rapide sur grand espace
from scipy.stats import randint
param_dist = {
    'n_estimators': randint(50, 500),
    'max_depth': randint(3, 20)
}
rand = RandomizedSearchCV(RandomForestClassifier(), param_dist, n_iter=50,
                        cv=5, random_state=42, n_jobs=-1)
rand.fit(X_train, y_train)
```

2.7 PIPELINES SKLEARN

Un Pipeline enchaîne les étapes de transformation et de modélisation de manière propre, évitant les fuites de données (data leakage).

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier

pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('classif', RandomForestClassifier(n_estimators=200))
])

pipe.fit(X_train, y_train)
y_pred = pipe.predict(X_test)

# Pipeline avec ColumnTransformer (features mixtes)
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder

preprocesseur = ColumnTransformer([
    ('num', StandardScaler(), cols_numeriques),
    ('cat', OneHotEncoder(), cols_categorielles)
])

pipe_complet = Pipeline([
    ('prep', preprocesseur),
    ('modele', RandomForestClassifier())
])

pipe_complet.fit(X_train, y_train)
```

RÈGLE D'OR : En production, toujours utiliser un Pipeline pour garantir

PARTIE III – KERAS & TENSORFLOW : FONDEMENTS DU DEEP LEARNING

3.1 KERAS : L'API HAUT NIVEAU DE TENSORFLOW

Keras est l'interface haut niveau de TensorFlow. Elle permet de construire des réseaux de neurones avec une syntaxe claire et intuitive.

DEUX APPROCHES PRINCIPALES :

- [A] SEQUENTIAL API – pour les architectures linéaires simples
- [B] FUNCTIONAL API – pour les architectures complexes (multi-entrées, multi-sorties, connexions résiduelles)

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
```

3.2 CONSTRUIRE UN RÉSEAU AVEC L'API SEQUENTIAL

```
# Réseau fully-connected pour classification binaire
modele = keras.Sequential([
    layers.Input(shape=(20,)),          # 20 features d'entrée
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.3),                # régularisation
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.2),
    layers.Dense(1, activation='sigmoid') # sortie binaire
])

modele.summary() # afficher l'architecture et le nombre de paramètres
```

3.3 COMPILATION : OPTIMISEUR, LOSS, MÉTRIQUES

La compilation définit comment le modèle va apprendre.

```
modele.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss='binary_crossentropy', # fonction de coût
    metrics=['accuracy', keras.metrics.AUC()]
)
```

CHOIX DE LA LOSS SELON LE PROBLÈME :

CLASSIFICATION BINAIRE	→ 'binary_crossentropy'
CLASSIFICATION MULTI-CLASS	→ 'sparse_categorical_crossentropy' (labels entiers)
	'categorical_crossentropy' (labels one-hot)
RÉGRESSION	→ 'mse' (Mean Squared Error)
	'mae' (Mean Absolute Error)
	'huber' (robuste aux outliers)

OPTIMISEURS COURANTS :

Adam	→ par défaut, polyvalent, adaptatif
SGD	→ stochastic gradient descent, avec momentum
RMSprop	→ bon pour RNN
AdamW	→ Adam + weight decay, excellent pour Transformers

3.4 ENTRAÎNEMENT ET CALLBACKS

```
historique = modele.fit(
    X_train, y_train,
    epochs=100,
    batch_size=32,
```



```

validation_data=(X_val, y_val),
callbacks=[
    keras.callbacks.EarlyStopping(
        monitor='val_loss',
        patience=10,
        restore_best_weights=True # restaurer le meilleur modèle
    ),
    keras.callbacks.ReduceLROnPlateau(
        monitor='val_loss',
        factor=0.5,
        patience=5,
        min_lr=1e-6
    ),
    keras.callbacks.ModelCheckpoint(
        filepath='meilleur_modele.keras',
        monitor='val_loss',
        save_best_only=True
    )
]
)

```

VISUALISATION DE L'ENTRAÎNEMENT :

```

plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(historique.history['loss'], label='Train Loss')
plt.plot(historique.history['val_loss'], label='Val Loss')
plt.title('Courbe de Loss')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(historique.history['accuracy'], label='Train Acc')
plt.plot(historique.history['val_accuracy'], label='Val Acc')
plt.title('Courbe d\'Accuraciy')
plt.legend()

plt.tight_layout()
plt.show()

```

3.5 L'API FONCTIONNELLE KERAS

Pour les architectures non-linéaires (résiduelles, multi-entrées, etc.) :

```

# Exemple : réseau résiduel simplifié
entrees = keras.Input(shape=(64,))
x = layers.Dense(128, activation='relu')(entrees)
x = layers.Dense(128, activation='relu')(x)

# Connexion résiduelle (skip connection)
shortcut = layers.Dense(128)(entrees)
x = layers.Add()([x, shortcut])
x = layers.Activation('relu')(x)

sorties = layers.Dense(10, activation='softmax')(x)

modele = keras.Model(inputs=entrees, outputs=sorties)
modele.summary()

```

3.6 COUCHES ESSENTIELLES KERAS

DENSE	→ couche fully-connected, paramètres : units, activation
DROPOUT	→ désactive aléatoirement des neurones (évite sur-apprentissage)
BATCHNORM	→ normalise les activations inter-couches, stabilise
CONV2D	→ convolution 2D pour images
MAXPOOLING2D	→ sous-échantillonnage spatial
FLATTEN	→ transforme tenseur ND en vecteur 1D
LSTM / GRU	→ mémoire pour séquences
EMBEDDING	→ représentation vectorielle de tokens/mots
ATTENTION	→ mécanisme d'attention (Transformers)
MULTIHEADATTENTION	→ attention multi-têtes (Transformers)

```
layers.BatchNormization()
layers.LayerNormalization() # préféré dans les Transformers
layers.GlobalAveragePooling2D()
layers.GlobalMaxPooling1D()
```

PARTIE IV – ALGORITHMES NATIFS DU DEEP LEARNING

4.1 LE NEURONE ARTIFICIEL : FONDEMENT DE TOUT

Un neurone artificiel calcule :

```
sortie = activation(  $\sum(\text{poids}_i \times \text{entrée}_i) + \text{biais}$  )
```

Le réseau apprend en ajustant les poids par rétropropagation du gradient (backpropagation).

4.2 LA DESCENTE DE GRADIENT STOCHASTIQUE (SGD)

Algorithme fondateur de l'apprentissage profond.

PRINCIPE :

1. Calculer la loss sur un mini-batch de données
2. Calculer le gradient de la loss par rapport aux poids
3. Mettre à jour les poids : $w = w - \text{lr} \times \nabla L(w)$
4. Répéter jusqu'à convergence

ADAM (Adaptive Moment Estimation) – version avancée du SGD :

- Maintient des moyennes mobiles du gradient (momentum)
- Maintient des moyennes mobiles du gradient au carré (variance)
- Taux d'apprentissage adaptatif par paramètre
- Default : $\text{lr}=0.001$, $\text{beta1}=0.9$, $\text{beta2}=0.999$, $\text{epsilon}=1e-8$

```
keras.optimizers.Adam(learning_rate=1e-3, beta_1=0.9, beta_2=0.999)
```

4.3 FONCTIONS D'ACTIVATION

ReLU (Rectified Linear Unit) $\rightarrow \max(0, x)$

- Standard pour couches cachées, rapide, évite vanishing gradient
- Problème : "dying ReLU" (neurones toujours à 0)

Leaky ReLU $\rightarrow \max(0.01x, x)$

- Résout le dying ReLU
- ```
layers.LeakyReLU(alpha=0.01)
```

ELU (Exponential Linear Unit)  $\rightarrow x$  si  $x > 0$  sinon  $\alpha(e^x - 1)$

- Sorties négatives centrées autour de 0

GELU (Gaussian Error Linear Unit)

- Utilisé dans BERT, GPT, Transformers
- Approximation :  $0.5 \times (1 + \tanh(\sqrt{2/\pi}(x + 0.044715x^3)))$

Sigmoid  $\rightarrow 1/(1+e^{-x}) \rightarrow$  sortie binaire (entre 0 et 1)

Tanh  $\rightarrow (e^x - e^{-x}) / (e^x + e^{-x}) \rightarrow$  centré en 0, pour RNN

Softmax  $\rightarrow$  normalisation probabiliste  $\rightarrow$  sortie multi-classe

RÈGLE : couches cachées  $\rightarrow$  ReLU/GELU | sortie binaire  $\rightarrow$  sigmoid |  
sortie multi-classe  $\rightarrow$  softmax | sortie régression  $\rightarrow$  linear

---

### 4.4 RÉGULARISATION EN DEEP LEARNING

---

DROPOUT :

```
layers.Dropout(rate=0.5)
```

- Pendant l'entraînement, désactive 50% des neurones aléatoirement

- Empêche la co-adaptation, force la robustesse

L2 REGULARIZATION (Weight Decay) :

```
layers.Dense(128, activation='relu',
 kernel_regularizer=keras.regularizers.l2(1e-4))
```

BATCH NORMALIZATION :

```
layers.BatchNormalization()
– Normalise les activations à chaque mini-batch
– Permet des learning rates plus élevés
– Réduit la sensibilité à l'initialisation
```

DATA AUGMENTATION (pour images) :

```
keras.layers.RandomFlip('horizontal')
keras.layers.RandomRotation(0.1)
keras.layers.RandomZoom(0.1)
```

---

## 4.5 AUTOENCODEURS (REPRÉSENTATION NON SUPERVISÉE)

---

Un autoencodeur apprend à compresser puis reconstruire les données.

Architecture : Encodeur → Espace latent (bottleneck) → Décodeur

# Autoencodeur simple

entree = keras.Input(shape=(784,)) # ex: image MNIST aplatie

# Encodeur

```
encoded = layers.Dense(128, activation='relu')(entree)
encoded = layers.Dense(64, activation='relu')(encoded)
latent = layers.Dense(32, activation='relu')(encoded) # espace latent
```

# Décodeur

```
decoded = layers.Dense(64, activation='relu')(latent)
decoded = layers.Dense(128, activation='relu')(decoded)
sortie = layers.Dense(784, activation='sigmoid')(decoded)
```

```
autoencoder = keras.Model(inputs=entree, outputs=sortie)
autoencoder.compile(optimizer='adam', loss='mse')
```

# L'encodeur seul pour extraction de features

```
encodeur = keras.Model(inputs=entree, outputs=latent)
```

APPLICATIONS :

- Réduction de dimension non-linéaire
- Détection d'anomalies (erreur de reconstruction élevée = anomalie)
- Débruitage (Denoising Autoencoder)
- Génération d'images via Variational Autoencoder (VAE)

VARIATIONAL AUTOENCODER (VAE) :

- Encodeur prédit  $\mu$  et  $\sigma^2$  (paramètres d'une distribution gaussienne)
- L'espace latent est une distribution continue → génération possible
- Loss = reconstruction loss + KL divergence

---

## PARTIE V – RÉSEAUX DE NEURONES CONVOLUTIFS (CNN)

---

---

### 5.1 POURQUOI LES CNN POUR LES IMAGES ?

---

Une image de 224×224×3 contient 150 528 pixels. Un Dense layer connecté à l'entrée aurait des millions de paramètres pour la première couche seulement.

Les CNN résolvent ce problème via :

- POIDS PARTAGÉS : le même filtre est appliqué partout sur l'image
- INVARIANCE LOCALE : détection de features indépendamment de leur position
- HIÉRARCHIE : couches basses → bords/textures, couches hautes → objets

---

### 5.2 OPÉRATION DE CONVOLUTION

---

Un filtre (kernel) de taille  $k \times k$  glisse sur l'image et calcule un produit scalaire à chaque position → Feature Map.

PARAMÈTRES CLÉS :

- filters : nombre de filtres (canaux de sortie)
- kernel\_size : taille du filtre (3×3, 5×5)
- strides : pas de déplacement (1 = dense, 2 = sous-échantillonnage)
- padding : 'same' (conserve dimensions) ou 'valid' (réduit dimensions)

```
layers.Conv2D(filters=32, kernel_size=(3,3), strides=(1,1),
 padding='same', activation='relu')
```

---

### 5.3 CNN COMPLET POUR CLASSIFICATION D'IMAGES

---

```
Exemple : classification CIFAR-10 (32x32x3, 10 classes)

modele = keras.Sequential([
 # Bloc 1
 layers.Conv2D(32, (3,3), padding='same', activation='relu',
 input_shape=(32, 32, 3)),
 layers.BatchNormalization(),
 layers.Conv2D(32, (3,3), padding='same', activation='relu'),
 layers.BatchNormalization(),
 layers.MaxPooling2D(pool_size=(2,2)),
 layers.Dropout(0.25),

 # Bloc 2
 layers.Conv2D(64, (3,3), padding='same', activation='relu'),
 layers.BatchNormalization(),
 layers.Conv2D(64, (3,3), padding='same', activation='relu'),
 layers.BatchNormalization(),
 layers.MaxPooling2D(pool_size=(2,2)),
 layers.Dropout(0.25),

 # Bloc 3
 layers.Conv2D(128, (3,3), padding='same', activation='relu'),
 layers.BatchNormalization(),
 layers.MaxPooling2D(pool_size=(2,2)),
 layers.Dropout(0.25),

 # Classifieur
 layers.Flatten(),
 layers.Dense(256, activation='relu'),
 layers.BatchNormalization(),
 layers.Dropout(0.5),
 layers.Dense(10, activation='softmax')
])

modele.compile(
 optimizer=keras.optimizers.Adam(1e-3),
 loss='sparse_categorical_crossentropy',
 metrics=['accuracy']
)
```

---

### 5.4 TRANSFER LEARNING : RÉUTILISER DES CNN PRÉ-ENTRAÎNÉS

---

Le Transfer Learning est l'une des techniques les plus puissantes du DL : réutiliser un modèle entraîné sur ImageNet (1.2M images) pour votre tâche.

```
from tensorflow.keras.applications import EfficientNetB0, ResNet50, VGG16

Charger le modèle de base sans la tête de classification
base = EfficientNetB0(
 include_top=False, # supprimer la couche Dense finale
 weights='imagenet', # poids pré-entraînés
 input_shape=(224, 224, 3)
)

ÉTAPE 1 : FEATURE EXTRACTION – geler les poids de base
base.trainable = False
```

```

Construire la nouvelle tête
entrees = keras.Input(shape=(224, 224, 3))
x = base(entrees, training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dropout(0.2)(x)
sorties = layers.Dense(nb_classes, activation='softmax')(x)

modele = keras.Model(inputs=entrees, outputs=sorties)

modele.compile(optimizer=keras.optimizers.Adam(1e-3),
 loss='sparse_categorical_crossentropy',
 metrics=['accuracy'])
modele.fit(X_train, y_train, epochs=10, validation_data=(X_val, y_val))

ÉTAPE 2 : FINE-TUNING – dégeler quelques couches hautes
base.trainable = True
for layer in base.layers[:-30]:
 layer.trainable = False # geler tout sauf les 30 dernières couches

modele.compile(optimizer=keras.optimizers.Adam(1e-5), # LR très faible !
 loss='sparse_categorical_crossentropy',
 metrics=['accuracy'])
modele.fit(X_train, y_train, epochs=10, validation_data=(X_val, y_val))

```

ARCHITECTURES DISPONIBLES DANS KERAS :

- VGG16, VGG19 → pionnières, lourdes
- ResNet50, ResNet101 → connexions résiduelles
- InceptionV3 → multi-scale
- MobileNetV2, V3 → léger, mobile
- EfficientNetB0 → B7 → état de l'art rapport perf/params
- EfficientNetV2 → encore plus rapide

---

## 5.5 CHARGEMENT DES IMAGES AVEC KERAS

---

```

Depuis un dossier organisé par classe
train_ds = keras.utils.image_dataset_from_directory(
 'data/train',
 image_size=(224, 224),
 batch_size=32,
 label_mode='int', # ou 'categorical', 'binary'
 shuffle=True
)

Préfetch pour performance
AUTOTUNE = tf.data.AUTOTUNE
train_ds = train_ds.prefetch(buffer_size=AUTOTUNE)

Augmentation des données intégrée
data_augmentation = keras.Sequential([
 layers.RandomFlip('horizontal'),
 layers.RandomRotation(0.15),
 layers.RandomZoom(0.15),
 layers.RandomBrightness(0.1)
])

```

---

## 5.6 DÉTECTION D'OBJETS (YOLO, SSD)

---

La détection d'objets localise ET classe plusieurs objets dans une image.

```

pip install ultralytics
from ultralytics import YOLO

Charger YOLOv8 pré-entraîné
model = YOLO('yolov8n.pt') # 'n' = nano (rapide), 's', 'm', 'l', 'x'

Inférence sur une image
results = model('mon_image.jpg')
results[0].show() # afficher les boîtes de détection

Fine-tuning sur données personnalisées (format YOLO)
model.train(

```

```

data='mon_dataset.yaml',
epochs=100,
imgsz=640,
batch=16
)

```

---

## PARTIE VI – RÉSEAUX RÉCURRENTS (RNN, LSTM, GRU)

---



---

### 6.1 POURQUOI LES RÉSEAUX RÉCURRENTS ?

---

Les réseaux Dense et CNN traitent des entrées de taille fixe sans mémoire.

Les RNN sont conçus pour les données séquentielles :

- Texte (séquence de mots/tokens)
- Séries temporelles (prix, capteurs, météo)
- Audio (séquence d'échantillons)
- ADN (séquence de bases)

Principe : à chaque pas de temps  $t$ , le réseau reçoit l'entrée  $x_t$  ET l'état caché  $h_{t-1}$  (mémoire du passé).

$$h_t = \text{activation}(W_h \times h_{t-1} + W_x \times x_t + b)$$


---

### 6.2 PROBLÈME DU VANISHING GRADIENT ET SOLUTION : LSTM

---

Le RNN simple souffre du vanishing gradient : les gradients s'évaporent pour les longues séquences → la mémoire à long terme est perdue.

LSTM (Long Short-Term Memory) résout ce problème via 3 portes :

- PORTE D'OUBLI (forget gate) : quoi oublier de la mémoire cellulaire
- PORTE D'ENTRÉE (input gate) : quoi stocker dans la mémoire cellulaire
- PORTE DE SORTIE (output gate): quoi exposer comme état caché

GRU (Gated Recurrent Unit) : version simplifiée du LSTM (2 portes), moins de paramètres, souvent aussi performant.

---

### 6.3 LSTM POUR PRÉVISION DE SÉRIES TEMPORELLES

---

```

import numpy as np
from tensorflow import keras
from tensorflow.keras import layers

Création des séquences : fenêtre glissante
def creer_sequences(serie, lookback=60):
 X, y = [], []
 for i in range(lookback, len(serie)):
 X.append(serie[i-lookback:i])
 y.append(serie[i])
 return np.array(X), np.array(y)

serie_normalisee : shape (N,) ou (N,1)
X, y = creer_sequences(serie_normalisee, lookback=60)
X.shape = (N-60, 60, 1) | y.shape = (N-60,)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
 shuffle=False)

Modèle LSTM
modele = keras.Sequential([
 layers.LSTM(128, return_sequences=True, input_shape=(60, 1)),
 layers.Dropout(0.2),
 layers.LSTM(64, return_sequences=False),
 layers.Dropout(0.2),
 layers.Dense(32, activation='relu'),
 layers.Dense(1)
])

```

```

modele.compile(optimizer='adam', loss='mse', metrics=['mae'])
modele.fit(X_train, y_train, epochs=50, batch_size=32,
 validation_data=(X_test, y_test))

```

PARAMÈTRE CLÉS :

return\_sequences=True → retourner la séquence complète (pour stacker LSTM)  
 return\_sequences=False → retourner seulement le dernier état (défaut)

---

## 6.4 RNN / LSTM POUR LE TRAITEMENT DU TEXTE (NLP CLASSIQUE)

---

```

from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

Tokenisation
tokenizer = Tokenizer(num_words=10000, oov_token='<OOV>')
tokenizer.fit_on_texts(textes_train)

Conversion texte → séquences d'entiers
sequences_train = tokenizer.texts_to_sequences(textes_train)
sequences_test = tokenizer.texts_to_sequences(textes_test)

Padding : aligner les longueurs
X_train = pad_sequences(sequences_train, maxlen=200, padding='post')
X_test = pad_sequences(sequences_test, maxlen=200, padding='post')

Modèle avec Embedding + BiLSTM
modele = keras.Sequential([
 layers.Embedding(input_dim=10000, output_dim=128, input_length=200),
 layers.Bidirectional(layers.LSTM(64, return_sequences=True)),
 layers.Bidirectional(layers.LSTM(32)),
 layers.Dense(64, activation='relu'),
 layers.Dropout(0.3),
 layers.Dense(1, activation='sigmoid') # classification binaire sentiments
])

modele.compile(optimizer='adam', loss='binary_crossentropy',
 metrics=['accuracy'])

```

NOTE : Pour les tâches NLP modernes, les Transformers (BERT, GPT) surpassent largement les LSTM. Utiliser les LSTM pour des cas simples ou contraintes de ressources.

---

## PARTIE VII – GENERATIVE ADVERSARIAL NETWORKS (GAN)

---



---

### 7.1 PHILOSOPHIE DES GAN

---

Inventés par Ian Goodfellow en 2014, les GAN sont composés de deux réseaux en compétition :

GÉNÉRATEUR (G) :

- Prend un vecteur de bruit aléatoire  $z$  (espace latent)
- Génère une donnée synthétique (image, texte, audio)
- Objectif : tromper le discriminateur

DISCRIMINATEUR (D) :

- Prend une donnée (réelle ou générée)
- Prédit si elle est réelle (1) ou fausse (0)
- Objectif : ne pas se laisser tromper

ÉQUILIBRE DE NASH : G et D améliorent mutuellement leurs compétences jusqu'à ce que G génère des données indiscernables des vraies données.

---

### 7.2 DCGAN : DEEP CONVOLUTIONAL GAN

---

```

import tensorflow as tf
from tensorflow import keras

```

```

from tensorflow.keras import layers

LATENT_DIM = 128

GÉNÉRATEUR : bruit → image
def build_generator():
 modele = keras.Sequential([
 layers.Dense(7 * 7 * 256, use_bias=False, input_shape=(LATENT_DIM,)),
 layers.BatchNormalization(),
 layers.LeakyReLU(0.2),
 layers.Reshape((7, 7, 256)),

 # Déconvolution (upsampling)
 layers.Conv2DTranspose(128, (5,5), strides=(1,1),
 padding='same', use_bias=False),
 layers.BatchNormalization(),
 layers.LeakyReLU(0.2),

 layers.Conv2DTranspose(64, (5,5), strides=(2,2),
 padding='same', use_bias=False),
 layers.BatchNormalization(),
 layers.LeakyReLU(0.2),

 layers.Conv2DTranspose(1, (5,5), strides=(2,2),
 padding='same', use_bias=False,
 activation='tanh')
])
 # Sortie : image 28x28x1 (MNIST), valeurs dans [-1, 1]
 return modele

DISCRIMINATEUR : image → probabilité réelle/fausse
def build_discriminateur():
 modele = keras.Sequential([
 layers.Conv2D(64, (5,5), strides=(2,2), padding='same',
 input_shape=(28,28,1)),
 layers.LeakyReLU(0.2),
 layers.Dropout(0.3),

 layers.Conv2D(128, (5,5), strides=(2,2), padding='same'),
 layers.LeakyReLU(0.2),
 layers.Dropout(0.3),

 layers.Flatten(),
 layers.Dense(1) # logit (pas de sigmoid ici, utilisé dans la loss)
])
 return modele

```

---

### 7.3 BOUCLE D'ENTRAÎNEMENT GAN

---

```

generator = build_generator()
discriminator = build_discriminateur()

cross_entropy = tf.keras.losses.BinaryCrossentropy(from_logits=True)

def discriminator_loss(real_output, fake_output):
 real_loss = cross_entropy(tf.ones_like(real_output), real_output)
 fake_loss = cross_entropy(tf.zeros_like(fake_output), fake_output)
 return real_loss + fake_loss

def generator_loss(fake_output):
 return cross_entropy(tf.ones_like(fake_output), fake_output)

gen_optimizer = tf.keras.optimizers.Adam(1e-4)
disc_optimizer = tf.keras.optimizers.Adam(1e-4)

@tf.function
def train_step(images):
 bruit = tf.random.normal([BATCH_SIZE, LATENT_DIM])

 with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
 images_generees = generator(bruit, training=True)

 real_output = discriminator(images, training=True)

```



```

fake_output = discriminator(images_generees, training=True)

gen_loss = generator_loss(fake_output)
disc_loss = discriminator_loss(real_output, fake_output)

gradients_gen = gen_tape.gradient(gen_loss,
 generator.trainable_variables)
gradients_disc = disc_tape.gradient(disc_loss,
 discriminator.trainable_variables)

gen_optimizer.apply_gradients(
 zip(gradients_gen, generator.trainable_variables))
disc_optimizer.apply_gradients(
 zip(gradients_disc, discriminator.trainable_variables))

return gen_loss, disc_loss

```

---

## 7.4 VARIANTES DE GAN ET PROBLÈMES COURANTS

---

### PROBLÈMES CLASSIQUES :

- MODE COLLAPSE : le générateur produit toujours les mêmes images
- INSTABILITÉ : les pertes oscillent sans converger
- DISPARITION GRAD : le discriminateur devient trop fort trop vite

### SOLUTIONS ET VARIANTES :

- |                        |                                                  |
|------------------------|--------------------------------------------------|
| WGAN (Wasserstein GAN) | → distance Wasserstein, plus stable              |
| WGAN-GP                | → gradient penalty, référence pour la stabilité  |
| StyleGAN2/3            | → génération d'images photo-réalistes (NVIDIA)   |
| CycleGAN               | → traduction image→image sans paires supervisées |
| Pix2Pix                | → traduction image→image avec paires supervisées |
| CGAN (Conditional GAN) | → génération conditionnée par un label           |
| BigGAN                 | → génération haute résolution, haute fidélité    |

### CONSEIL PRATIQUE :

- Commencer avec WGAN-GP plutôt que GAN standard
- Utiliser Spectral Normalization dans le discriminateur
- Ne pas mettre Batch Normalization dans le discriminateur
- Utiliser LeakyReLU (alpha=0.2) partout sauf sortie générateur (tanh)

---

## PARTIE VIII – LATENT DIFFUSION MODELS (LDM)

---



---

### 8.1 DES GAN AUX MODÈLES DE DIFFUSION

---

Les Diffusion Models (2020+) ont supplanté les GAN pour la génération d'images grâce à une stabilité d'entraînement bien supérieure et une qualité d'image exceptionnelle.

#### PRINCIPE DU PROCESSUS DE DIFFUSION :

- FORWARD : ajouter progressivement du bruit gaussien à une image (T étapes, ex: T=1000) jusqu'à obtenir du bruit pur
- BACKWARD : entraîner un réseau de neurones à débruiter pas à pas, apprenant ainsi à reconstruire des images depuis le bruit

---

### 8.2 LATENT DIFFUSION MODEL : L'INNOVATION CLÉ

---

Les LDM (Rombach et al., 2022 – base de Stable Diffusion) résolvent le coût computationnel en opérant dans l'ESPACE LATENT plutôt que l'espace pixel.

#### ARCHITECTURE D'UN LDM :

- [1] AUTOENCODEUR VARIATIONNEL (VAE)
  - Encodeur : image haute résolution → espace latent compressé (ex: 64×4)
  - Décodeur : espace latent → image reconstruite
  - Entraîné séparément, indépendant du processus de diffusion
- [2] U-NET DÉBRUITEUR
  - Réseau en forme de U avec skip connections

- Prédit le bruit ajouté à chaque étape  $t$
- Inclut des couches d'attention (Self-Attention, Cross-Attention)
- C'est le cœur du processus de diffusion

### [3] ENCODEUR DE TEXTE (CLIP / T5)

- Convertit le prompt texte en embeddings
- Ces embeddings guident la génération via Cross-Attention dans le U-Net

---

## 8.3 UTILISER STABLE DIFFUSION AVEC PYTHON

---

```
pip install diffusers transformers accelerate
from diffusers import StableDiffusionPipeline
import torch

Charger le pipeline
pipe = StableDiffusionPipeline.from_pretrained(
 "runwayml/stable-diffusion-v1-5",
 torch_dtype=torch.float16 # demi-précision pour économiser VRAM
)
pipe = pipe.to("cuda")

Génération d'image
prompt = "a futuristic city at sunset, cyberpunk, detailed, 8k"
negative_prompt = "blurry, low quality, deformed"

image = pipe(
 prompt=prompt,
 negative_prompt=negative_prompt,
 num_inference_steps=50, # étapes de débruitage (20-50)
 guidance_scale=7.5, # adhérence au prompt (7-12)
 width=512, height=512
).images[0]

image.save("generation.png")
```

---

## 8.4 FINE-TUNING D'UN LDM : DREAMBOOTH ET LORA

---

### DREAMBOOTH :

- Fine-tune tout le modèle sur ~5-20 images personnalisées
- Associe un concept rare (ex: "sks chien") à ces images
- Lourd en VRAM (~24 Go), long à entraîner

### LoRA (Low-Rank Adaptation) :

- Fine-tune seulement des matrices de faible rang ajoutées au U-Net
- Beaucoup plus léger (fichier de quelques centaines de Mo)
- Se combine avec le modèle de base au moment de l'inférence

```
pip install peft
from peft import LoraConfig, get_peft_model

lora_config = LoraConfig(
 r=4, # rang des matrices LoRA
 lora_alpha=32,
 target_modules=['to_q', 'to_v'], # modules à adapter
 lora_dropout=0.05
)
UNET_lora = get_peft_model(unet, lora_config)
```

### SCHEDULERS DE BRUIT (SAMPLER) :

```
from diffusers import DDIMScheduler, PNDMScheduler, DPMSolverMultistepScheduler

pipe.scheduler = DPMSolverMultistepScheduler.from_config(pipe.scheduler.config)
DPM-Solver++ → très rapide (20 étapes suffisent), haute qualité
```

---

## 8.5 ARCHITECTURE U-NET AVEC ATTENTION

---

```
U-Net simplifié pour diffusion (conceptuel)

class UNetBlock(keras.layers.Layer):
```

```

def __init__(self, filters, **kwargs):
 super().__init__(**kwargs)
 self.conv1 = layers.Conv2D(filters, 3, padding='same')
 self.conv2 = layers.Conv2D(filters, 3, padding='same')
 self.norm1 = layers.LayerNormalization()
 self.norm2 = layers.LayerNormalization()
 self.act = layers.Activation('swish')

def call(self, x, training=False):
 h = self.norm1(x)
 h = self.act(self.conv1(h))
 h = self.norm2(h)
 h = self.conv2(h)
 return x + h # connexion résiduelle

Le U-Net complet ajoute :
- Couche de Self-Attention entre les blocs
- Injection du timestep t via sinusoidal embedding + couche Dense
- Cross-Attention avec les embeddings texte (pour génération conditionnée)

```

---

## PARTIE IX – TRANSFORMERS ET LARGE LANGUAGE MODELS (LLM)

---



---

### 9.1 LA RÉVOLUTION TRANSFORMER (2017)

---

Le papier "Attention Is All You Need" (Vaswani et al., 2017) a révolutionné le Deep Learning en proposant une architecture sans récurrence, basée entièrement sur le mécanisme d'attention.

AVANTAGES SUR LES RNN :

- Parallélisation totale de l'entraînement (vs séquentiel pour RNN)
- Capture les dépendances à longue distance efficacement
- Scalabilité : plus le modèle est grand, plus il est performant

---

### 9.2 LE MÉCANISME D'ATTENTION

---

L'attention permet à chaque token de "regarder" tous les autres tokens et de pondérer leur importance.

SCALED DOT-PRODUCT ATTENTION :

$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \times V$

- Q (Query) : "Que suis-je en train de chercher ?"
- K (Key) : "Que contient chaque position ?"
- V (Value) : "Quelle information retourner ?"
- $d_k$  : dimension des clés (facteur d'échelle)

```

Implémentation Keras
attention_layer = layers.MultiHeadAttention(
 num_heads=8,
 key_dim=64, # dimension par tête
 dropout=0.1
)
output, weights = attention_layer(
 query=x, key=x, value=x, # self-attention
 return_attention_scores=True
)

```

MULTI-HEAD ATTENTION :

- h têtes d'attention en parallèle, chacune apprend différentes relations
- Résultats concaténés et projetés linéairement
- h=8 ou 12 typiquement

---

### 9.3 ARCHITECTURE TRANSFORMER ENCODER

---

```

def transformer_encoder_block(embed_dim, num_heads, ff_dim, dropout=0.1):

```

```

entrees = keras.Input(shape=(None, embed_dim))

Multi-Head Self-Attention
x = layers.LayerNormalization(epsilon=1e-6)(entrees)
x = layers.MultiHeadAttention(num_heads=num_heads,
 key_dim=embed_dim//num_heads,
 dropout=dropout)(x, x)
x = layers.Dropout(dropout)(x)
x = layers.Add()([entrees, x]) # connexion résiduelle

Feed-Forward Network
residuel = x
x = layers.LayerNormalization(epsilon=1e-6)(x)
x = layers.Dense(ff_dim, activation='gelu')(x)
x = layers.Dropout(dropout)(x)
x = layers.Dense(embed_dim)(x)
x = layers.Dropout(dropout)(x)
x = layers.Add()([residuel, x]) # connexion résiduelle

return keras.Model(inputs=entrees, outputs=x)

```

#### POSITIONAL ENCODING :

Les Transformers n'ont pas de sens de l'ordre naturel → on injecte des informations de position via encodage sinusoïdal.

```

def positional_encoding(max_len, d_model):
 positions = np.arange(max_len)[: , np.newaxis]
 dims = np.arange(d_model)[np.newaxis, :]
 angles = positions / np.power(10000, (2*(dims//2)) / np.float32(d_model))
 angles[:, 0::2] = np.sin(angles[:, 0::2]) # dimensions paires → sin
 angles[:, 1::2] = np.cos(angles[:, 1::2]) # dimensions impaires → cos
 return tf.cast(angles[np.newaxis, :, :], dtype=tf.float32)

```

---

## 9.4 BERT : ENCODER BIDIRECTIONNEL

---

BERT (Bidirectional Encoder Representations from Transformers, 2018) utilise uniquement l'encodeur Transformer et prédit dans les deux directions.

#### PRÉ-ENTRAÎNEMENT :

- Masked Language Modeling (MLM) : prédire les tokens masqués [MASK]
- Next Sentence Prediction (NSP) : prédire si deux phrases se suivent

#### UTILISATION VIA HUGGING FACE :

```

pip install transformers
from transformers import AutoTokenizer, AutoModel
import torch

tokenizer = AutoTokenizer.from_pretrained('bert-base-uncased')
modele_bert = AutoModel.from_pretrained('bert-base-uncased')

texte = "The movie was absolutely fantastic and I loved it!"
tokens = tokenizer(texte, return_tensors='pt', padding=True,
 truncation=True, max_length=128)

with torch.no_grad():
 outputs = modele_bert(**tokens)

CLS token → représentation de la phrase entière
cls_embedding = outputs.last_hidden_state[:, 0, :]
token_embeddings → représentation de chaque token
token_embeddings = outputs.last_hidden_state

```

---

## 9.5 FINE-TUNING DE BERT POUR CLASSIFICATION DE TEXTE

---

```

from transformers import AutoTokenizer, AutoModelForSequenceClassification
from transformers import TrainingArguments, Trainer
from datasets import Dataset
import torch

Préparer le dataset

```

```

data = {'text': textes, 'label': labels}
dataset = Dataset.from_dict(data)
dataset = dataset.train_test_split(test_size=0.2)

Tokenisation du dataset
tokenizer = AutoTokenizer.from_pretrained('bert-base-uncased')

def tokenize(batch):
 return tokenizer(batch['text'], padding=True,
 truncation=True, max_length=128)

dataset = dataset.map(tokenize, batched=True)

Modèle pré-entraîné + tête de classification
modele = AutoModelForSequenceClassification.from_pretrained(
 'bert-base-uncased', num_labels=2
)

Arguments d'entraînement
args = TrainingArguments(
 output_dir='./bert_finetune',
 num_train_epochs=3,
 per_device_train_batch_size=16,
 per_device_eval_batch_size=32,
 warmup_steps=200,
 weight_decay=0.01,
 learning_rate=2e-5,
 evaluation_strategy='epoch',
 save_strategy='epoch',
 load_best_model_at_end=True
)

trainer = Trainer(
 model=modele,
 args=args,
 train_dataset=dataset['train'],
 eval_dataset=dataset['test']
)
trainer.train()

```

---

## 9.6 GPT : DECODER AUTORÉGRESSIF

---

GPT (Generative Pre-trained Transformer) utilise uniquement le décodeur Transformer avec attention causale (masquée) : chaque token ne "voit" que les tokens précédents.

PRÉ-ENTRAÎNEMENT : Next Token Prediction sur des milliards de tokens.

ARCHITECTURE DECODER :

- Même structure que l'encodeur MAIS avec Masked Self-Attention
- Le masque causal empêche de voir le futur pendant l'entraînement

UTILISATION AVEC HUGGING FACE :

```

from transformers import AutoTokenizer, AutoModelForCausalLM
import torch

tokenizer = AutoTokenizer.from_pretrained('gpt2')
modele_gpt = AutoModelForCausalLM.from_pretrained('gpt2')

input_text = "La science des données est"
inputs = tokenizer(input_text, return_tensors='pt')

with torch.no_grad():
 outputs = modele_gpt.generate(
 **inputs,
 max_new_tokens=100,
 do_sample=True,
 temperature=0.8, # 0 = déterministe, 1 = aléatoire
 top_k=50, # considérer seulement les 50 meilleurs tokens
 top_p=0.9, # nucleus sampling
 repetition_penalty=1.2
)

```

```
print(tokenizer.decode(outputs[0], skip_special_tokens=True))
```

---

## 9.7 LES GRANDS LLM : LANDSCAPE ACTUEL

---

### MODÈLES OPEN SOURCE :

|                |                                                      |
|----------------|------------------------------------------------------|
| Llama 3 (Meta) | → très performant, plusieurs tailles (8B, 70B, 405B) |
| Mistral 7B     | → compact et efficace, excellent rapport perf/taille |
| Mixtral 8x7B   | → architecture MoE (Mixture of Experts)              |
| Falcon         | → entraîné sur de grandes données web                |
| Gemma (Google) | → léger, disponible en 2B et 7B                      |

### MODÈLES PROPRIÉTAIRES :

|                     |                             |
|---------------------|-----------------------------|
| GPT-4o (OpenAI)     | → multimodal, référence     |
| Claude (Anthropic)  | → raisonnement, sécurité    |
| Gemini (Google)     | → multimodal, contexte long |
| Command R+ (Cohere) | → optimisé RAG              |

### INFÉRENCE LOCALE AVEC OLLAMA :

```
Installer ollama, puis :
ollama pull llama3
ollama run llama3
```

```
import requests
```

```
def query_ollama(prompt, model='llama3'):
 response = requests.post(
 'http://localhost:11434/api/generate',
 json={'model': model, 'prompt': prompt, 'stream': False}
)
 return response.json()['response']
```

```
reponse = query_ollama("Explique la descente de gradient en 3 lignes.")
print(reponse)
```

---

## 9.8 RAG : RETRIEVAL AUGMENTED GENERATION

---

Le RAG enrichit un LLM avec des documents externes sans le fine-tuner.

### PIPELINE RAG COMPLET :

- [1] INGESTION : charger les documents, les découper en chunks
- [2] EMBEDDING : convertir chaque chunk en vecteur dense
- [3] INDEXATION : stocker les vecteurs dans une base vectorielle
- [4] RETRIEVAL : pour une question, récupérer les chunks les plus similaires
- [5] GÉNÉRATION : injecter les chunks comme contexte dans le prompt LLM

```
pip install langchain chromadb sentence-transformers
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.embeddings import HuggingFaceEmbeddings
from langchain.vectorstores import Chroma
from langchain.llms import Ollama
from langchain.chains import RetrievalQA
```

```
Splitter
splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
chunks = splitter.split_documents(documents)
```

```
Embeddings
embeddings = HuggingFaceEmbeddings(
 model_name='sentence-transformers/all-MiniLM-L6-v2'
)
```

```
Vectorstore
vectorstore = Chroma.from_documents(chunks, embeddings,
 persist_directory='./chroma_db')
```

```
Pipeline QA
llm = Ollama(model='llama3')
qa_chain = RetrievalQA.from_chain_type(
 llm=llm,
 retriever=vectorstore.as_retriever(search_kwargs={'k': 4}))
```

```
)
reponse = qa_chain.run("Quelle est la différence entre LSTM et GRU ?")
print(reponse)
```

---

## 9.9 FINE-TUNING LLM AVEC QLoRA

---

QLoRA (Quantized LoRA) : fine-tune un LLM quantifié en 4-bits avec LoRA.  
Permet d'adapter LLama 7B sur une GPU de 16 Go.

```
pip install transformers peft bitsandbytes trl datasets

from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig
from peft import LoraConfig, get_peft_model, prepare_model_for_kbit_training
from trl import SFTTrainer, TrainingArguments

Quantification 4-bits
bnb_config = BitsAndBytesConfig(
 load_in_4bit=True,
 bnb_4bit_quant_type='nf4',
 bnb_4bit_compute_dtype=torch.bfloat16,
 bnb_4bit_use_double_quant=True
)

model = AutoModelForCausalLM.from_pretrained(
 'meta-llama/Llama-3-8b',
 quantization_config=bnb_config,
 device_map='auto'
)

model = prepare_model_for_kbit_training(model)

lora_config = LoraConfig(
 r=16, lora_alpha=32,
 target_modules=['q_proj', 'v_proj', 'k_proj', 'o_proj'],
 lora_dropout=0.05, bias='none',
 task_type='CAUSAL_LM'
)

model = get_peft_model(model, lora_config)
model.print_trainable_parameters() # ~1% des paramètres seulement !

trainer = SFTTrainer(
 model=model,
 train_dataset=dataset,
 dataset_text_field='text',
 max_seq_length=1024,
 args=TrainingArguments(
 output_dir='./qlora_output',
 num_train_epochs=3,
 per_device_train_batch_size=4,
 gradient_accumulation_steps=4,
 learning_rate=2e-4,
 fp16=True,
 logging_steps=10
)
)

trainer.train()
```

---

## PARTIE X – PIPELINES, DÉPLOIEMENT ET BONNES PRATIQUES

---

### 10.1 SAUVEGARDE ET CHARGEMENT DES MODÈLES

---

KERAS :

```
Sauvegarde complète (architecture + poids + optimiseur)
modele.save('mon_modele.keras')

Chargement
modele_charge = keras.models.load_model('mon_modele.keras')
```

```
Sauvegarde des poids uniquement
modele.save_weights('poids.weights.h5')
modele.load_weights('poids.weights.h5')

SKLEARN :
import joblib

joblib.dump(pipeline, 'pipeline_ml.joblib')
pipeline_charge = joblib.load('pipeline_ml.joblib')

PYTORCH :
torch.save(model.state_dict(), 'modele.pth')
model.load_state_dict(torch.load('modele.pth'))
```

---

## 10.2 SERVIR UN MODÈLE AVEC FASTAPI

---

```
pip install fastapi uvicorn pydantic
from fastapi import FastAPI
from pydantic import BaseModel
import numpy as np
import joblib

app = FastAPI(title="API ML GCI")
modele = joblib.load('pipeline_ml.joblib')

class InputData(BaseModel):
 features: list[float]

class Prediction(BaseModel):
 classe: int
 probabilite: float

@app.post('/predict', response_model=Prediction)
def predict(data: InputData):
 X = np.array(data.features).reshape(1, -1)
 classe = int(modele.predict(X)[0])
 proba = float(modele.predict_proba(X)[0].max())
 return Prediction(classe=classe, probabilite=proba)

Lancer avec : uvicorn main:app --host 0.0.0.0 --port 8000 --reload
```

---

## 10.3 BONNES PRATIQUES INCONTOURNABLES

---

- [1] REPRODUCTIBILITÉ  
Fixer les seeds partout :  

```
np.random.seed(42)
tf.random.set_seed(42)
torch.manual_seed(42)
```
- [2] SPLIT CHRONOLOGIQUE (séries temporelles)  
Ne jamais mélanger train/test sur données temporelles → fuite future
- [3] NORMALISATION  
Toujours fitter le scaler sur train, transformer train ET test  
Ne jamais fitter sur test ou l'ensemble complet
- [4] BASELINE D'ABORD  
Commencer par le modèle le plus simple (régression logistique, DummyClassifier) avant complexité → comprendre l'apport réel
- [5] MONITORING DU DRIFT  
Les données changent dans le temps → surveiller la distribution des données d'inférence vs données d'entraînement (Evidently AI)
- [6] DOCUMENTATION DES EXPÉRIENCES  
Utiliser MLflow ou Weights & Biases pour tracer chaque expérience : hyperparamètres, métriques, artefacts  
  

```
import mlflow

with mlflow.start_run():
```



```

mlflow.log_param('lr', 0.001)
mlflow.log_param('epochs', 50)
mlflow.log_metric('val_accuracy', 0.923)
mlflow.keras.log_model(modele, 'model')

```

#### [7] EXPLIQUER LES PRÉDICTIONS

Utiliser SHAP pour l'interprétabilité :

```

pip install shap

```

```

import shap
explainer = shap.TreeExplainer(model_xgb)
shap_values = explainer.shap_values(X_test)
shap.summary_plot(shap_values, X_test, feature_names=feature_names)

```

---

## PARTIE XI – PROJETS COMPLETS COMMENTÉS

---



---

### 11.1 PROJET 1 : CLASSIFICATION DE TEXTE AVEC BERT (SENTIMENT)

---

"""

Objectif : classifier des avis en positif/négatif avec BERT fine-tuné  
Dataset : IMDB, Amazon reviews, ou données propriétaires

"""

```

from datasets import load_dataset
from transformers import (AutoTokenizer, AutoModelForSequenceClassification,
 TrainingArguments, Trainer)

import numpy as np
from sklearn.metrics import accuracy_score, f1_score

```

```

1. Charger le dataset
dataset = load_dataset('imdb')

```

```

2. Tokenisation
tokenizer = AutoTokenizer.from_pretrained('distilbert-base-uncased')

```

```

def preprocess(batch):
 return tokenizer(batch['text'], truncation=True, padding='max_length',
 max_length=256)

```

```

dataset = dataset.map(preprocess, batched=True)
dataset = dataset.rename_column('label', 'labels')
dataset.set_format(type='torch', columns=['input_ids', 'attention_mask',
 'labels'])

```

```

3. Modèle
modele = AutoModelForSequenceClassification.from_pretrained(
 'distilbert-base-uncased', num_labels=2
)

```

```

4. Métriques
def compute_metrics(eval_pred):
 logits, labels = eval_pred
 predictions = np.argmax(logits, axis=-1)
 return {
 'accuracy': accuracy_score(labels, predictions),
 'f1': f1_score(labels, predictions, average='weighted')
 }

```

```

5. Entraînement
args = TrainingArguments(
 output_dir='./sentiment_bert',
 evaluation_strategy='epoch',
 num_train_epochs=3,
 per_device_train_batch_size=32,
 per_device_eval_batch_size=64,
 learning_rate=2e-5,
 weight_decay=0.01,
 load_best_model_at_end=True,
 metric_for_best_model='f1'
)

```

```

trainer = Trainer(
 model=model,
 args=args,
 train_dataset=dataset['train'],
 eval_dataset=dataset['test'],
 compute_metrics=compute_metrics
)
trainer.train()

6. Inférence
def predire_sentiment(texte):
 tokens = tokenizer(texte, return_tensors='pt', truncation=True,
 max_length=256)
 with torch.no_grad():
 logits = modele(**tokens).logits
 classe = torch.argmax(logits).item()
 return 'Positif' if classe == 1 else 'Négatif'

print(predire_sentiment("This film was absolutely amazing!"))
→ Positif

```

---

## 11.2 PROJET 2 : DÉTECTION D'ANOMALIES AVEC AUTOENCODEUR

---

```

"""
Objectif : détecter les transactions frauduleuses (anomalies rares)
Idée : entraîner l'autoencodeur sur les transactions normales.
 Une transaction anormale aura une erreur de reconstruction élevée.
"""

import numpy as np
import pandas as pd
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.preprocessing import StandardScaler

Supposons X_normal = transactions normales uniquement (train)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_normal)
X_test_scaled = scaler.transform(X_test_all)

n_features = X_train_scaled.shape[1]

Architecture autoencodeur
entree = keras.Input(shape=(n_features,))
encode = layers.Dense(64, activation='relu')(entree)
encode = layers.Dense(32, activation='relu')(encode)
latent = layers.Dense(16, activation='relu')(encode)
decode = layers.Dense(32, activation='relu')(latent)
decode = layers.Dense(64, activation='relu')(decode)
sortie = layers.Dense(n_features, activation='linear')(decode)

ae = keras.Model(inputs=entree, outputs=sortie)
ae.compile(optimizer='adam', loss='mse')

ae.fit(X_train_scaled, X_train_scaled, epochs=100, batch_size=256,
 validation_split=0.1,
 callbacks=[keras.callbacks.EarlyStopping(patience=10)])

Calcul de l'erreur de reconstruction sur le set de test
X_recon = ae.predict(X_test_scaled)
erreurs = np.mean(np.square(X_test_scaled - X_recon), axis=1)

Seuil : percentile 95 sur les données d'entraînement
X_train_recon = ae.predict(X_train_scaled)
erreurs_train = np.mean(np.square(X_train_scaled - X_train_recon), axis=1)
seuil = np.percentile(erreurs_train, 95)

anomalies = erreurs > seuil
print(f"Anomalies détectées : {anomalies.sum()} / {len(anomalies)}")

```

---

## 11.3 PROJET 3 : CNN POUR CLASSIFICATION D'IMAGES MÉDICALES

---

---

```

"""
Objectif : classifier des images radiologiques (normal vs pathologie)
Approche : Transfer Learning avec EfficientNetB3
"""

from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.applications import EfficientNetB3

IMG_SIZE = 300
NB_CLASSES = 2
BATCH_SIZE = 32

Chargement des données
train_ds = keras.utils.image_dataset_from_directory(
 'data/train', image_size=(IMG_SIZE, IMG_SIZE), batch_size=BATCH_SIZE)
val_ds = keras.utils.image_dataset_from_directory(
 'data/val', image_size=(IMG_SIZE, IMG_SIZE), batch_size=BATCH_SIZE)

Normalisation et augmentation
normalisation = layers.Rescaling(1./255)
augmentation = keras.Sequential([
 layers.RandomFlip('horizontal'),
 layers.RandomRotation(0.1),
 layers.RandomZoom(0.1),
 layers.RandomContrast(0.1)
])

AUTOTUNE = tf.data.AUTOTUNE
train_ds = train_ds.map(lambda x, y: (augmentation(normalisation(x)), y))
train_ds = train_ds.prefetch(AUTOTUNE)
val_ds = val_ds.map(lambda x, y: (normalisation(x), y)).prefetch(AUTOTUNE)

Modèle Transfer Learning
base = EfficientNetB3(include_top=False, weights='imagenet',
 input_shape=(IMG_SIZE, IMG_SIZE, 3))
base.trainable = False

entrees = keras.Input(shape=(IMG_SIZE, IMG_SIZE, 3))
x = base(entrees, training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dense(256, activation='relu')(x)
x = layers.BatchNormalization()(x)
x = layers.Dropout(0.4)(x)
sorties = layers.Dense(NB_CLASSES, activation='softmax')(x)

modele = keras.Model(inputs=entrees, outputs=sorties)

Phase 1 : Feature extraction
modele.compile(optimizer=keras.optimizers.Adam(1e-3),
 loss='sparse_categorical_crossentropy',
 metrics=['accuracy'])
modele.fit(train_ds, validation_data=val_ds, epochs=20,
 callbacks=[keras.callbacks.EarlyStopping(patience=5,
 restore_best_weights=True)])

Phase 2 : Fine-tuning
base.trainable = True
for layer in base.layers[:-50]:
 layer.trainable = False

modele.compile(optimizer=keras.optimizers.Adam(1e-5),
 loss='sparse_categorical_crossentropy',
 metrics=['accuracy'])
modele.fit(train_ds, validation_data=val_ds, epochs=20,
 callbacks=[keras.callbacks.EarlyStopping(patience=5,
 restore_best_weights=True)])

modele.save('classification_medicale.keras')

```

---

#### 11.4 RÉCAPITULATIF : CHOISIR SON ARCHITECTURE

---

| PROBLÈME                        | ARCHITECTURE RECOMMANDÉE                  |
|---------------------------------|-------------------------------------------|
| Données tabulaires, < 1M lignes | Scikit-learn : RF, XGBoost, SVR           |
| Données tabulaires, > 1M lignes | LightGBM, XGBoost avec early stopping     |
| Classification d'images         | CNN (EfficientNet, ResNet) + Transfer     |
| Détection d'objets              | YOLOv8, EfficientDet                      |
| Séries temporelles courtes      | LSTM, GRU, 1D-CNN                         |
| Séries temporelles longues      | Transformer (Temporal Fusion Transformer) |
| Classification de texte         | BERT / DistilBERT fine-tuné               |
| Génération de texte             | GPT-2 fine-tuné ou Llama via LoRA         |
| Q&A sur documents privés        | RAG (LLM + VectorDB)                      |
| Génération d'images             | Stable Diffusion (LDM)                    |
| Données synthétiques (images)   | StyleGAN3, WGAN-GP                        |
| Détection d'anomalies           | Autoencodeur, Isolation Forest            |
| Clustering                      | K-Means, DBSCAN, HDBSCAN                  |

## CONCLUSION – LA VOIE DU MAÎTRE EN ML/DL

Le Machine Learning et le Deep Learning ne sont pas des baguettes magiques. Ils sont des outils. La valeur vient de la rigueur du praticien, pas du modèle.

### PRINCIPE 1 – TOUJOURS COMMENCER PAR UN BASELINE SIMPLE

Un modèle simple qui fonctionne vaut mieux qu'un modèle complexe mystérieux.

### PRINCIPE 2 – LES DONNÉES DOMINENT

80% du travail est dans les données. Un bon dataset avec un modèle moyen surpasse un mauvais dataset avec le meilleur modèle.

### PRINCIPE 3 – COMPRENDRE AVANT D'OPTIMISER

Avant de tuner, comprendre pourquoi le modèle se trompe.  
La matrice de confusion, les courbes de loss et SHAP sont vos alliés.

### PRINCIPE 4 – REPRODUIRE AVANT D'INNOVER

Implémenter les architectures classiques soi-même avant d'utiliser les librairies. Comprendre les mécanismes fondamentaux.

### PRINCIPE 5 – DÉPLOYER OU CE N'EST PAS DU ML

Un modèle non déployé n'a aucune valeur. Apprendre FastAPI, Docker, la surveillance de modèles en production.

### PRINCIPE 6 – L'IA ÉVOLUE VITE, LES FONDAMENTAUX NON

Les architectures changent. Les mathématiques, la rigueur statistique, et la pensée algorithmique restent.

"Un senior ML/DL ne sait pas tout. Il sait chercher, comprendre, adapter et valider rigoureusement. Et il sait quand ne PAS utiliser le Deep Learning."

– GCI, Département Formation IA

GCI – GYKHAMINE CONCEPT INVESTIGATION

FIN DU COURS – ML & DEEP LEARNING EN PYTHON

© GCI – Usage interne – Formation & Intelligence Artificielle